Voici l'ensemble de la stack technique d'ODSM-v2 (système de gestion scolaire — ODSM), basé sur package.json, mobile/pubspec.yaml, prisma/schema.prisma et la structure src/.

🖥️ Web (application principale)

Framework — Next.js 16.1.6 (App Router, RSC) avec React 19.2, TypeScript 5, build Turbopack. Mode hybride : pages SSR + API routes.

UI — Tailwind CSS v4, shadcn/ui (composants Radix UI : dialog, select, accordion, popover…), framer-motion (animations), recharts (graphiques), lucide-react (icônes), sonner (toasts), cmdk (command palette), react-day-picker + date-fns.

Formulaires — react-hook-form + zod v4 (validation).

Auth & sécurité — src/lib/auth.ts : sessions via cookie HTTP-only chiffré (src/lib/crypto.ts), 2FA via otplib + génération de QR codes (qrcode), mots de passe hachés avec bcryptjs, rôles/garde par accès (role-permissions.ts, middleware.ts).

🗄️ Données

Base de données — SQLite (better-sqlite3 v12) via Prisma 7 (adapter @prisma/adapter-better-sqlite3, src/lib/db.ts). Schéma de 30+ modèles dans prisma/schema.prisma (School, User, Student, Teacher, Grade, Assessment, LessonPlan, ClassSession, Payment, License, AnnualDecision…). IDs UUID. Config de l'école stockée en JSON (colonnes String @default("{}")).

📱 Applications mobiles (deux approches)

1. App Flutter — dossier mobile/ (app odsm_mobile) : Dart, Material 3, sqflite (base locale), http (API REST), provider (state), shared_preferences, crypto, intl. Cross-platform (Android/iOS/Windows/Linux/macOS/web). Fonctionnement offline : découvre le serveur sur le réseau local (scan du subnet 10.222.62.X via GET /api/discovery), pull/push des données via /api/sync/pull et /api/sync/push, mise en cache dans SQLite (database_service.dart).

2. App Web embarquée via Capacitor — dossier android/ + capacitor.config.ts : Capacitor 8 (appId: com.odsm.app) pour empaqueter la web app en APK Android.

🌐 Backend

API REST — Routes Next.js dans src/app/api/ : admin/, school/, staff/, teacher/, sync/pull + sync/push, discovery/, debug/, testdb/. Synchronisation de données entre serveur et appareils mobiles hors-ligne.

🛠️ Outillage

- Prisma CLI + scripts de migration (migrate-to-uuid.ts, migrate-assessments.ts)

- ESLint 9 (eslint-config-next), shadcn CLI

- Scripts Node/PowerShell de diagnostic et maintenance (check_db.js, inspect_db.js, deep_cleanup.ts, sync-prod.ps1)

- 2FA, licences (license-check.ts), année scolaire (school-year-utils.ts), numérotation élèves/profs (student-numbering.ts), bulletins MINEDUC (post-cteb-nomenclature.ts, maxima-utils.ts)

En résumé : Next.js 16 + React 19 + Prisma/SQLite + Tailwind/shadcn au cœur, avec Flutter et Capacitor comme clients mobiles.